Lesson 13 - Homework

Error Handling - Xətaların idarə olunması

Mövzu xatırlatmaları

Python proqramında xəta baş verdikdə proqramın tamamilə dayanmasının qarşısını almaq üçün `try / except` istifadə edə bilərik.

Əsas struktur

```
try:
    # Xəta yarada biləcək kod

except ValueError:
    # ValueError baş verdikdə işləyəcək kod
```

`try`

Xəta yarana biləcək kod `try` daxilində yazılır.

```
try:
    yas = int(input("Yaşınızı daxil edin: "))
```

`except`

Xəta baş verdikdə proqramın nə etməli olduğunu müəyyən edir.

```
try:
    yas = int(input("Yaşınızı daxil edin: "))

except ValueError:
    print("Yaş rəqəm olmalıdır.")
```

Mümkün qədər:

```
except:
```

əvəzinə konkret exception yazmağa çalışın:

```
except ValueError:
```

Bir neçə `except`

Bir `try` blokunda fərqli xətaları ayrıca idarə edə bilərik.

```
try:
    reqem1 = int(input("Birinci rəqəm: "))
    reqem2 = int(input("İkinci rəqəm: "))

    netice = reqem1 / reqem2

except ValueError:
    print("Yalnız rəqəm daxil edin.")

except ZeroDivisionError:
    print("Sıfıra bölmək olmaz.")
```

`else`

`try` daxilində heç bir xəta baş verməzsə `else` işləyir.

```
try:
    yas = int(input("Yaş: "))

except ValueError:
    print("Yanlış məlumat.")

else:
    print("Məlumat uğurla qəbul edildi.")
```

`finally`

`finally` xəta olsa da, olmasa da işləyir.

```
try:
    yas = int(input("Yaş: "))

except ValueError:
    print("Yanlış məlumat.")

finally:
    print("əməliyyat tamamlandı.")
```

`raise`

Bəzən Python özü xəta yaratmır, amma daxil edilən məlumat bizim proqramın qaydalarına uyğun olmur.

Bu zaman özümüz exception yarada bilərik.

```
yas = int(input("Yaş: "))

if yas < 0:
    raise ValueError("Yaş mənfi ola bilməz.")
```

Exception mesajını əldə etmək

```
try:
    yas = int(input("Yaş: "))

except ValueError as xeta:
    print(f"Xəta: {xeta}")
```

File Operations ilə birlikdə

```
try:
    with open("users.txt", "r", encoding="utf-8") as fayl:
        melumat = fayl.read()

except FileNotFoundError:
    print("Fayl tapılmadı.")
```

Tapşırıq - Mağaza sifariş sistemi

Bu homework boyunca eyni proqram üzərində işləyəcəksiniz.

Hər yeni mərhələdə əvvəlki kodunuzu silməyin.

Əvvəlki tapşırıqda yazdığınız proqramı inkişaf etdirin və yeni funksionallıq əlavə edin.

Başlanğıc məhsullarımız:

```
mehsullar = {
    "laptop": 1800,
    "telefon": 1200,
    "monitor": 500,
    "mouse": 50
}
```

Mərhələ 1 - Məhsul və miqdarın alınması

İlk olaraq sadə sifariş proqramı yaradın.

İstifadəçidən:

```
Məhsul:
Miqdar:
```

məlumatlarını alın.

Məhsulun qiymətini `mehsullar` dictionary-sindən əldə edin.

Sonra ümumi qiyməti hesablayın:

```
ümumi qiymət = məhsulun qiyməti × miqdar
```

Məsələn:

```
Məhsul: monitor
Miqdar: 2

Məhsul: monitor
Miqdar: 2
Ümumi qiymət: 1000 AZN
```

Bu mərhələdə

Hələ error handling əlavə etməyin.

Əvvəlcə normal işləyən proqram yaradın.

Mərhələ 2 - Miqdar üçün `ValueError`

İndi əvvəlki proqramınızı inkişaf etdirin.

Hazırda istifadəçi belə bir məlumat daxil etsə:

```
Miqdar: iki
```

proqram xəta verəcək.

Miqdarı `try / except` daxilində `int` tipinə çevirməyə çalışın.

Əgər istifadəçi rəqəm əvəzinə mətn daxil edərsə:

```
Miqdar rəqəm olmalıdır.
```

mesajı göstərilməlidir.

İstifadə edin

```
try
except ValueError
```

Test edin

```
Məhsul: monitor
Miqdar: iki
```

Proqram crash etməməlidir.

Mərhələ 3 - Miqdarın düzgünlüyünün yoxlanılması

İndi proqram başqa bir problem yaşayır.

İstifadəçi:

```
Miqdar: -3
```

yaza bilər.

Python üçün `-3` düzgün integer-dir, ona görə `ValueError` avtomatik yaranmayacaq.

Amma bizim mağazamız üçün mənfi miqdar düzgün deyil.

Əgər miqdar `0` və ya mənfidirsə özünüz exception yaradın.

```
raise ValueError(...)
```

Mesaj:

```
Miqdar 0-dan böyük olmalıdır.
```

Test 1

```
Məhsul: monitor
Miqdar: -2
```

Test 2

```
Məhsul: monitor
Miqdar: 0
```

Hər iki halda proqram sifariş yaratmamalıdır.

Mərhələ 4 - Mövcud olmayan məhsul

İndi məhsul hissəsini də təhlükəsiz edin.

Hazırda istifadəçi:

```
Məhsul: televizor
```

yazarsa, dictionary-də belə key olmadığı üçün proqram:

```
KeyError
```

verəcək.

Bu xətanı idarə edin.

Məhsulu belə əldə etməyə çalışın:

```
qiymet = mehsullar[mehsul]
```

və `KeyError` baş verdikdə:

```
Belə məhsul mövcud deyil.
```

göstərin.

Şərt

Bu mərhələdə:

```
mehsullar.get(...)
```

istifadə etməyin.

Məqsəd KeyError exception-unu idarə etməkdir.

Test

```
Məhsul: televizor
Miqdar: 2
```

Nəticə:

```
Belə məhsul mövcud deyil.
```

Mərhələ 5 - Bir neçə exception-un idarə olunması

Artıq proqramda ən azı iki fərqli xəta yarana bilər:

```
ValueError
KeyError
```

Kodunuzu elə qurun ki, bunların hər biri ayrıca idarə olunsun.

Struktur təxminən belə olmalıdır:

```
try:
    # məhsulun tapılması
    # miqdarın alınması
    # yoxlamalar
    # qiymətin hesablanması

except KeyError:
    # məhsul tapılmadıqda

except ValueError as xeta:
    # miqdar düzgün olmadıqda
```

Test 1

```
Məhsul: televizor
Miqdar: 2
```

Test 2

```
Məhsul: monitor
Miqdar: üç
```

Test 3

```
Məhsul: monitor
Miqdar: -4
```

Test 4

```
Məhsul: monitor
Miqdar: 3
```

Dörd fərqli vəziyyətdə proqram düzgün davranmalıdır.

Mərhələ 6 - `else` əlavə edin

İndi uğurlu sifariş mesajını `try` blokunun içində göstərməyin.

Bunun üçün `else` istifadə edin.

Əgər heç bir exception baş verməzsə:

```
Sifariş uğurla yaradıldı.

Məhsul: monitor
Miqdar: 2
Ümumi qiymət: 1000 AZN
```

göstərilməlidir.

Məqsəd

Flow belə olmalıdır:

```
Xəta var → except

Xəta yoxdur → else
```

Mərhələ 7 - Sifarişi fayla yazın

İndi əvvəlki proqramınızı bir az da inkişaf etdirin.

Sifariş uğurla yaradıldıqda məlumatı:

```
orders.txt
```

faylına yazın.

Append mode istifadə edin:

```
"a"
```

Format:

```
monitor,2,1000
mouse,3,150
laptop,1,1800
```

Hər yeni sifariş yeni sətirdə olmalıdır.

Nümunə

İstifadəçi:

```
Məhsul: mouse
Miqdar: 3
```

daxil edirsə, fayla:

```
mouse,3,150
```

əlavə olunmalıdır.

Vacib

Səhv məlumat daxil edilərsə fayla heç nə yazılmamalıdır.

Mərhələ 8 - Sifariş tarixçəsini göstərmək

İndi proqramın sonuna yeni hissə əlavə edin.

Sifariş yaradıldıqdan sonra:

```
orders.txt
```

faylını oxuyun və bütün əvvəlki sifarişləri göstərin.

Məsələn:

```
--- Sifariş tarixçəsi ---

monitor,2,1000
mouse,3,150
laptop,1,1800
```

Amma bir problem var.

Əgər proqram ilk dəfə işləyirsə və:

```
orders.txt
```

hələ mövcud deyilsə, proqram `FileNotFoundError` verə bilər.

Bu xətanı da idarə edin.

Mesaj:

```
Hələ heç bir sifariş tarixçəsi yoxdur.
```

İstifadə edin

```
except FileNotFoundError:
```

Mərhələ 9 - `finally` əlavə edin

Proqramın əsas sifariş əməliyyatının sonunda `finally` istifadə edin.

Sifariş uğurlu olsa da, xəta baş versə də:

```
Sifariş əməliyyatı tamamlandı.
```

mesajı göstərilməlidir.

Xatırlatma

`finally`:

- xəta olduqda işləyir;
- xəta olmadıqda da işləyir.

Final nəticə

Homework-un sonunda proqramınız aşağıdakı işləri bacarmalıdır:

1. İstifadəçidən məhsul adı alır.
2. Məhsulu dictionary-də axtarır.
3. Mövcud olmayan məhsulu `KeyError` ilə idarə edir.
4. Miqdarı istifadəçidən alır.
5. Miqdar rəqəm deyilsə `ValueError` idarə edir.
6. Miqdar `0` və ya mənfidirsə `raise` istifadə edir.
7. Ümumi qiyməti hesablayır.
8. Uğurlu nəticəni `else` daxilində göstərir.
9. Sifarişi `orders.txt` faylına əlavə edir.
10. Sifariş tarixçəsini fayldan oxuyur.
11. Fayl mövcud deyilsə `FileNotFoundError` idarə edir.
12. `finally` istifadə edərək əməliyyatın bitdiyini göstərir.